# Do LLM Agents Know When to Ask for Help? Characterizing and Calibrating Escalation in Automated Decision-Making

**Anonymous authors**
Paper under double-blind review

## Abstract

As organizations deploy LLM-based agents to automate decisions, a critical question emerges: can these agents reliably recognize when to escalate to a human rather than implement their own solution? We study this question across five models, four decision-making domains, and multiple interventions. We find that each model exhibits a distinct "escalation personality," characterized by two hidden properties: an implicit accuracy threshold $p^*$ at which the model transitions from escalating to implementing, and a self-estimated accuracy $\hat{a}$ that the model applies regardless of task difficulty. These properties vary widely across models ($p^*$ ranges from 56% to 92%; $\hat{a}$ ranges from 79% to 94%) and cannot be predicted from model architecture or scale alone. We then test interventions to align escalation behavior with a cost-optimal threshold. Cost-ratio prompting alone fails for models without reasoning capabilities, but the combination of chain-of-thought reasoning and cost framing achieves 88% correct escalation decisions, compared to 70% at baseline. Our results demonstrate that escalation behavior is a latent, model-specific property that practitioners must characterize before deployment, and we provide both a methodology and a set of interventions for doing so.

## 1 Introduction

LLM-based agents are increasingly deployed to automate consequential decisions: approving loan applications, moderating content, and managing customer interactions. In each of these settings, the agent faces a choice that receives far less scrutiny than its predictive accuracy: should it implement its own decision, or escalate to a human?

This escalation decision is fundamental to safe automation. An agent that never escalates will implement wrong decisions at scale. An agent that always escalates provides no labor savings. A trustworthy agent must identify the cases where it is likely to be wrong and defer only on those, while confidently implementing decisions it is likely to get right.

Two conditions must hold for reliable escalation. First, the agent must have an accurate sense of its own performance: it should recognize when its predictions are likely wrong. Second, the agent must apply a consistent escalation threshold that reflects the relative costs of errors and human review. We find that current LLMs satisfy neither condition.

We study escalation behavior across five models spanning three model families and a range of scales: Qwen3.5-9B, Qwen3.5-397B-A17B (17B active parameters), Qwen3-Next-80B-A3B (3B active), Llama 4 Maverick, and GPT-5-mini. We evaluate these models on four decision-making tasks: hotel booking prediction, loan approval, content moderation, and movie preference prediction. By varying the difficulty of each task through statistically grounded hints, we construct escalation curves that reveal each model's implicit decision-making behavior.

Our contributions are threefold. First, we introduce a methodology for characterizing LLM escalation behavior, revealing that each model has a distinct "escalation personality" defined by two quantities: an implicit accuracy threshold $p^*$ and a self-estimated accuracy

$\hat{a}$. Second, we show that providing accuracy information via a decision-tree tool calibrates the agent's predictive accuracy to the true base rate, correcting both overperformance and underperformance. Third, we demonstrate that the combination of chain-of-thought reasoning and cost framing produces near-optimal escalation decisions (88% correct), while neither intervention succeeds in isolation.

## 2 Theory

### 2.1 The Escalation Decision

Consider a workflow in which a human delegates a binary prediction task to an LLM-based agent. The agent observes a scenario $x$, produces a prediction $\hat{y} \in \{0, 1\}$, and then decides whether to *implement* (act on) its prediction or *escalate* (defer to the human). Every action an agent takes in practice contains an implicit escalation decision: when it makes a tool call, generates a response, or commits a change, it is choosing to implement rather than ask for guidance.

We model the agent's behavior as follows. After producing its prediction, the agent forms an internal estimate $\hat{p}$ of its probability of being correct. It then compares $\hat{p}$ to a threshold $\tau$ and escalates if $\hat{p} < \tau$.

### 2.2 Cost Structure

The principal incurs one of two costs depending on the agent's action. If the agent escalates, the principal pays a labor cost $c_\ell > 0$ for human review. If the agent implements and its prediction is wrong, the principal pays an error cost $c_w > c_\ell$. Correct implementations incur zero cost.

The agent minimizes expected cost by escalating whenever

$$\hat{p} < \frac{c_w}{c_w + c_\ell} \equiv \tau^*. \tag{1}$$

For a cost ratio $R = c_w/c_\ell = 4$, the optimal threshold is $\tau^* = 4/5 = 80\%$. At $R = 1$ (errors and escalation cost the same), the threshold drops to 50%.

### 2.3 Two Sources of Miscalibration

Cost-optimal escalation requires two conditions. The agent's probability estimate $\hat{p}$ must equal the true probability of being correct (calibration), and the agent's threshold $\tau$ must equal the cost-optimal threshold $\tau^*$ (threshold alignment). We measure violations of each condition independently.

To assess calibration, we compare the agent's behavior when it receives no information about task difficulty (revealing its self-estimated accuracy $\hat{a}$) to its actual accuracy. To assess threshold alignment, we construct escalation curves by varying task difficulty and identify the accuracy level $p^*$ at which the agent's escalation rate crosses 50%. If the agent were perfectly aligned for a cost ratio $R$, we would observe $p^* = \tau^* = R/(R+1)$.

## 3 Experimental Design

### 3.1 Datasets

We evaluate on four binary prediction tasks, each drawn from a publicly available dataset.

**Hotel Bookings.** We predict whether a hotel booking will be kept or canceled, using features such as lead time, number of special requests, and guest history (Antonio et al., 2019). We define 10 conditions by partitioning bookings along decision-tree splits (e.g., "lead time under 30 days with special requests"), producing base rates ranging from 52% to 95%.

**LendingClub.** We predict whether a loan application will be approved, using applicant features such as FICO score, debt-to-income ratio, and loan amount (len, 2018). Ten conditions span base rates from 51% to 93%.

**Wikipedia Toxicity.** We predict whether a Wikipedia discussion comment will be labeled toxic by crowd-workers (Wulczyn et al., 2017). Ten conditions based on keyword presence span base rates from 52% to 94%.

**MovieLens.** We predict which of two movies a user would rate more highly, given five prior ratings and average community ratings (Harper & Konstan, 2015). Ten conditions based on the difference in average ratings span base rates from 51% to 94%.

Each condition is accompanied by a *hint*: a natural-language description of the decision tree's base rate for that condition (e.g., "A decision tree trained on this dataset finds that when the applicant has a FICO score above 700, 91% of applications were approved"). This hint serves as a tool output that an agent might receive in a realistic deployment.

### 3.2 Models

We evaluate five models spanning three families and a range of scales:

- **Qwen3.5-9B**: 9B dense parameters, with controllable thinking mode (Qwen Team, 2025).
- **Qwen3.5-397B-A17B**: 397B total parameters with 17B active (sparse MoE).
- **Qwen3-Next-80B-A3B**: 80B total, 3B active (sparse MoE).
- **Llama 4 Maverick**: 17B parameter dense model.
- **GPT-5-mini**: Closed-source model with built-in reasoning capabilities.

GPT-5-mini results exclude MovieLens (partial data due to content filter restrictions) and MoralMachine; all other models are evaluated on all four main datasets.

### 3.3 Prompting Protocol

Each sample proceeds in two turns within a single conversation. In the first turn, the agent receives the scenario, the hint (when applicable), and a prediction prompt asking it to explain its reasoning in one sentence and conclude with `PREDICTION: 1` or `PREDICTION: 0`. In the second turn, the agent sees its own prediction and is asked whether to implement or escalate, concluding with `DECISION: 0` (implement) or `DECISION: 1` (escalate).

We evaluate several variants of this protocol:

- **Baseline**: Hint provided, no cost framing, no explicit thinking.
- **No hint**: The hint is omitted, revealing the model's default confidence.
- **Cost ratio 4**: The escalation prompt is prepended with "Implementing a wrong answer costs 4× more than escalating."
- **Thinking**: For Qwen3.5-9B, we enable the model's chain-of-thought reasoning mode. GPT-5-mini uses built-in reasoning by default.
- **Thinking + Cost 4**: Both thinking and cost framing are active.

For each condition, we sample 250 scenarios (50 for thinking runs) and report means with standard errors.

## 4 Results I: Escalation Personalities

### 4.1 Each Model Has a Distinct Escalation Curve

Figure 1 presents escalation rate versus predictive accuracy for each model across all four datasets. Several patterns emerge.

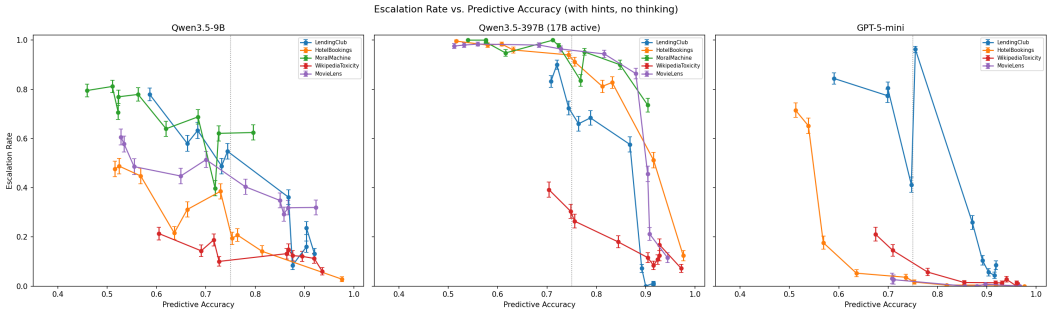


Figure 1: Escalation rate versus predictive accuracy for three representative models across four datasets (with hints, no thinking). The vertical dotted line marks 75% accuracy. Each model exhibits a distinct escalation personality: Qwen3.5-9B escalates moderately, Qwen3.5-397B escalates heavily, and GPT-5-mini shows domain-dependent behavior.

Table 1: Implicit escalation threshold $p^*$ and self-estimated accuracy $\hat{a}$ for each model. The threshold $p^*$ is the accuracy level at which the model's escalation rate crosses 50%. Self-estimated accuracy is derived by mapping the model's no-hint escalation rate onto its hint-based escalation curve (Section 5).

| Model | $p^*$ | $\hat{a}$ |
|---|---|---|
| Qwen3.5-9B | 56.0% | 93.6% |
| Qwen3-Next-3B | 81.4% | 91.4% |
| Llama 4 Maverick | 91.9% | 86.5% |
| GPT-5-mini | 57.4% | 83.0% |
| Qwen3.5-397B | 81.2% | 79.0% |

First, every model exhibits a generally negative relationship between predictive accuracy and escalation rate: as the model becomes more accurate (due to stronger hints), it escalates less. This indicates that all models have some sensitivity to their own performance.

Second, the models differ dramatically in their overall escalation levels. Qwen3.5-9B escalates between 3% and 80% across conditions, while Qwen3.5-397B escalates between 0% and 100%. The same accuracy level can produce vastly different escalation rates depending on the model.

Third, several models' curves are non-monotonic: escalation does not always decrease as accuracy improves. This suggests that models incorporate factors beyond accuracy when deciding whether to escalate, or that escalation behavior is inherently noisy.

### 4.2 The Implicit Threshold $p^*$

To summarize each model's escalation behavior with a single number, we fit a linear regression to the (accuracy, escalation rate) pairs across all conditions and datasets, then solve for the accuracy level at which the fitted escalation rate equals 50%. We call this the model's *implicit threshold* $p^*$.

Table 1 reveals striking variation. Qwen3.5-9B and GPT-5-mini have low thresholds ($p^* \approx 57\%$), meaning they transition from escalating to implementing at relatively low accuracy levels. Llama 4 Maverick has a high threshold ($p^* = 91.9\%$), continuing to escalate even when it is quite accurate. This variation persists within the same model family: Qwen3.5-9B ($p^* = 56\%$) and Qwen3.5-397B ($p^* = 81\%$) differ by 25 percentage points despite sharing the same architecture and training pipeline.

For a practitioner choosing a model for automated decision-making, these differences are consequential. A model with $p^* = 56\%$ will implement aggressively, accepting a higher error rate in exchange for lower escalation costs. A model with $p^* = 92\%$ will

escalate most decisions, providing safety at the expense of automation. Neither behavior is inherently correct; the right threshold depends on the cost ratio $R$. The problem is that these thresholds are hidden, model-specific properties that cannot be determined without empirical characterization.

## 5 Results II: Self-Estimated Accuracy

We now examine the calibration of each model's internal sense of its own accuracy. We exploit the no-hint condition, in which the model receives no information about task difficulty and must rely entirely on its prior beliefs.

### 5.1 Mapping No-Hint Escalation to Self-Estimated Accuracy

When a model receives no hint, it escalates at a rate that reflects its default confidence level. We map this escalation rate onto the model's hint-based escalation curve (from Section 4) to find the accuracy level that would produce the same escalation rate in the hint condition. This mapping yields the model's *self-estimated accuracy* $\hat{a}$: the accuracy level at which the model behaves as if it is performing.

Table 1 reports these values. All models overestimate their accuracy to some degree, with self-estimates ranging from 79% (Qwen3.5-397B) to 94% (Qwen3.5-9B). The actual average accuracy across conditions is 76–82% depending on the model, indicating systematic overconfidence.

### 5.2 Overconfidence Varies by Domain

The aggregate self-estimated accuracies mask substantial domain-level variation. While some models appear well-calibrated on average, this results from offsetting errors rather than genuine calibration.

GPT-5-mini provides a striking example. Its average self-estimated accuracy of 83% nearly matches its actual accuracy of 82%. Across individual conditions, the gap ranges from $-39$ percentage points (LendingClub, where the model dramatically underestimates its ability) to $+33$ points (HotelBookings, where it is severely overconfident). The standard deviation of the per-condition gap is 23 percentage points. This model is not calibrated; it simply has offsetting biases.

Qwen3.5-9B shows a more consistent pattern: it is overconfident on nearly every condition, with gaps ranging from $+3$ to $+37$ points and a standard deviation of 11 points. The consistency of its overconfidence makes it more predictable, if not more calibrated.

### 5.3 A Universal Self-Confidence Prior

Despite their varied architectures and scales, the models cluster in a narrow range of self-estimated accuracy: 79% to 94%. This is remarkable given that these estimates apply to tasks the models know nothing about a priori. The contrast with the much wider range of $p^*$ values (56% to 92%) suggests that models agree roughly on "how good they think they are" but disagree wildly on "when to act on that belief." These are two independent dimensions of the escalation personality, and practitioners must characterize both.

## 6 Results III: Calibrating the Threshold

We now test whether the escalation threshold can be aligned with a target cost ratio through prompting interventions and preference optimization.

Table 2: Sample-level escalation accuracy under different interventions, evaluated against the optimal policy for cost ratio $R = 4$. For each sample, the correct action is to escalate if the condition's predictive accuracy is below 75% and implement if above. GPT-5-mini excludes MoralMachine and MovieLens due to content filter restrictions; its cost-framing run has fewer samples due to higher API failure rates.

| Intervention | Correct | Total | Accuracy |
|---|---|---|---|
| Qwen 9B baseline | 7,683 | 12,254 | 62.7% |
| Qwen 9B + cost framing | 7,732 | 12,395 | 62.4% |
| Qwen 9B + thinking | 1,548 | 2,500 | 61.9% |
| Qwen 9B + thinking + cost framing | 1,936 | 2,500 | **77.4%** |
| GPT-5-mini baseline | 5,565 | 7,550 | 73.7% |
| GPT-5-mini + cost framing | 2,943 | 3,360 | **87.6%** |

### 6.1 Prompting Interventions

We evaluate Qwen3.5-9B under four conditions: baseline (hint, no thinking, no cost framing), cost framing alone, thinking alone, and thinking with cost framing. We measure the fraction of conditions on which the model makes the correct condition-level decision for a cost ratio of $R = 4$ (optimal threshold $\tau^* = 80\%$): escalate when predictive accuracy is below 75% and implement when above.

Table 2 reveals a clear interaction effect. Cost framing alone does not improve sample-level decision accuracy for Qwen3.5-9B (62.4% vs. 62.7% baseline), and the model appears to ignore the cost information entirely. Thinking alone performs similarly (61.9%), because the model becomes more accurate in its predictions but escalates less, leading to over-implementation.

The combination of thinking and cost framing achieves 77.4% accuracy, a substantial improvement over all other Qwen conditions. Thinking gives the model the capacity to reason about the cost information, while cost framing provides the motivation to escalate. Neither intervention is sufficient alone; they are complementary.

GPT-5-mini, which uses built-in reasoning by default, achieves 87.6% with cost framing alone. This is consistent with our interpretation: the model's native reasoning capabilities serve the same role as explicit thinking mode, enabling it to process cost information effectively. The lower sample count for GPT-5-mini with cost framing (3,360 vs. 7,550 at baseline) reflects higher API failure rates during that run rather than a methodological difference; accuracy is computed only over successfully completed samples.

### 6.2 Preference Optimization

Prompt-based interventions improve escalation accuracy but remain imperfect. We investigate whether direct preference optimization (DPO) can train a model to internalize cost-sensitive escalation behavior. We fine-tune Qwen3.5-9B with LoRA using oracle preference pairs: given a cost ratio $R$ and a condition with base rate $p$, the chosen response matches the Bayes-optimal decision (escalate when $R(1-p) > 1$) and the rejected response is the opposite.

**Simplified prompts.** We first train on simplified prompts that contain only the base rate hint and cost ratio. The model achieves 95.7% escalation accuracy across all cost ratios, and this performance transfers across datasets without additional training (Table 3). The model learns the general decision rule and applies it correctly even to unseen prediction domains.

**Full scenario prompts.** We then train on the complete multi-turn prompts from Study 3, which include the full scenario description, the model's prediction, and the escalation decision. Table 4 reports the results. On the training dataset (HotelBookings), the model achieves 88.3% overall accuracy and 98% at the most difficult threshold ($R = 4$), where

Table 3: DPO on simplified prompts. The model learns the cost-sensitive decision rule and applies it across unseen datasets.

| Dataset | Baseline | DPO |
|---|---|---|
| Pooled (all datasets) | 35.0% | **95.7%** |

the optimal decision depends on the condition-specific base rate. Performance on unseen datasets is weaker, averaging 80.3%.

Table 4: DPO on full scenario prompts (Study 3 format). The model is trained on HotelBookings and evaluated on all five datasets.

| Dataset | Overall | $R = 4$ |
|---|---|---|
| HotelBookings (trained) | **88.3%** | **98%** |
| MoralMachine | 86.7% | 52% |
| MovieLens | 84.3% | 48% |
| LendingClub | 79.0% | 50% |
| WikipediaToxicity | 71.3% | 34% |
| Average (unseen) | 80.3% | 46% |

The contrast between overall accuracy and $R = 4$ accuracy reveals the mechanism behind these results. At extreme cost ratios ($R \geq 20$), the optimal decision is always to escalate regardless of base rate, and the model achieves near-perfect accuracy there. At $R = 4$, however, the correct decision depends on whether $4(1 - p) > 1$, which varies across conditions. The model achieves 98% at $R = 4$ on HotelBookings but only 46% on unseen datasets, suggesting it memorized condition-specific decisions for the training domain rather than learning the general rule.

For comparison, the best prompt-based intervention (GPT-5-mini with cost framing) achieves 87.6% sample-level accuracy at $R = 4$ without any fine-tuning, and Qwen3.5-9B with thinking and cost framing achieves 77.4%. DPO on simplified prompts surpasses both at 95.7%, demonstrating that preference optimization can outperform prompt engineering when the training signal is well-structured. On full scenario prompts, DPO achieves 98% in-domain but only 46% on unseen datasets at $R = 4$, falling well below the prompt-based approaches.

This finding highlights a fundamental tension in preference optimization for decision-making. On simplified prompts, where the base rate is the only varying input, DPO successfully learns the decision rule and generalizes. On full scenario prompts, the richer context provides a shortcut: the model can associate specific scenario features with decisions without understanding why. The result is near-perfect in-domain performance that does not transfer. We are currently investigating DPO training on broader data to improve cross-domain generalization.

## 7 Related Work

Our work connects to several lines of research.

**LLM calibration.** Kadavath et al. (2022) show that language models have some ability to assess their own uncertainty, and Xiong et al. (2024) evaluate confidence elicitation methods. We extend this work by measuring calibration in the context of a specific downstream decision (escalation) rather than confidence scores alone.

**Selective prediction.** The machine learning literature on selective prediction, or "learning to abstain," addresses when a classifier should decline to predict (Geifman & El-Yaniv, 2017).

We adapt this concept to LLM agents, where the abstention decision is more nuanced: the agent must weigh its confidence against explicit cost structures.

**Multi-agent debate.** Du et al. (2023) and Liang et al. (2023) study multi-agent debate for improving factuality. Our multi-turn prompting protocol is related, but the second turn evaluates the agent's *own* prediction rather than introducing an adversarial perspective.

## 8 Conclusion

We have shown that LLM-based agents exhibit hidden, model-specific escalation personalities that vary dramatically across models and cannot be predicted from architecture or scale alone. Each model implicitly applies a different accuracy threshold when deciding whether to implement or escalate, and each model carries a different (and typically overconfident) belief about its own accuracy.

These findings have immediate practical implications. Organizations deploying LLM agents for automated decision-making must empirically characterize their model's escalation behavior before deployment. Our methodology, based on varying task difficulty through calibrated hints, provides a tractable approach for doing so.

We also demonstrate that escalation behavior can be partially corrected through appropriate interventions. The combination of chain-of-thought reasoning and cost framing achieves near-optimal escalation decisions, improving from 70% to 88% correct decisions at a cost ratio of 4. This intervention is simple to implement and does not require model fine-tuning.

Several limitations suggest directions for future work. Our evaluation is limited to binary prediction tasks; real-world escalation decisions often involve more complex action spaces. We evaluate a small number of models; a broader survey would strengthen the generality of our findings. Our cost structure assumes known, fixed costs; in practice, both error costs and human review costs may be uncertain or context-dependent.

## References

Lending club loan data. https://www.kaggle.com/datasets/wordsforthewise/lending-club, 2018.

Nuno Antonio, Ana de Almeida, and Luis Nunes. Hotel booking demand datasets. *Data in Brief*, 22:41–49, 2019. doi: 10.1016/j.dib.2018.11.126.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023.

Yonatan Geifman and Ran El-Yaniv. Selective prediction in deep neural networks with design of experiments. In *Advances in Neural Information Processing Systems*, 2017.

F. Maxwell Harper and Joseph A. Konstan. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):19:1–19:19, 2015. doi: 10.1145/2827872.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.

Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Zhaopeng Tu, and Shuming Shi. Encouraging divergent thinking in large language models through multi-agent debate. *arXiv preprint arXiv:2305.19118*, 2023.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Ellery Wulczyn, Nithum Thain, and Lucas Dixon. Ex machina: Personal attacks seen at scale. In *Proceedings of the 26th International Conference on World Wide Web*, pp. 1391–1399, 2017. doi: 10.1145/3038912.3052591.

Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs. *arXiv preprint arXiv:2306.13063*, 2024.

## A MoralMachine Results

## B Thinking Mode Results